

ESTRUTURAS DE SELEÇÃO

Tiago Henrique Angioletti





ESTRUTURAS DE SELEÇÃO

Mecanismos fundamentais para controlar o fluxo de um programa.
Tomam decisões com base em condições lógicas.



OPERADORES DE COMPARAÇÃO

São utilizados para comparar valores e expressões.

Resultam em um valor booleano (verdadeiro ou falso).



== (IGUAL A)

Verifica se os dois valores são iguais

```
int idade = 18;  
bool ehMaiorDeIdade = idade == 18; // Retorna verdadeiro
```



!= (DIFERENTE DE)

Verifica se dois valores são diferentes

```
double saldoAtual = 500.0;  
bool precisaRecarregar = saldoAtual != 0.0; // Retorna verdadeiro
```



> (MAIOR QUE)

Verifica se o valor da esquerda é maior do que o valor da direita.

```
int nota = 85;  
bool passou = nota > 60; // Retorna verdadeiro
```



<> (MENOR QUE)

Verifica se o valor da esquerda é menor do que o valor da direita.

```
int nota = 40;  
bool reprovou = nota < 60; // Retorna verdadeiro
```



>= (MAIOR OU IGUAL A)

Verifica se o valor da esquerda é maior ou igual ao valor da direita.

```
int nota = 60;  
bool passou = nota >= 60; // Retorna verdadeiro
```




<= (MENOR OU IGUAL A)

Verifica se o valor da esquerda é menor ou igual ao valor da direita.

```
int nota = 59;  
bool reprovou = nota <= 59; // Retorna verdadeiro
```



MANTENHA A LÓGICA CLARA

Use parênteses para evitar ambiguidades em expressões complexas.

CONSISTÊNCIA DE TIPOS

Os valores comparados devem ser do mesmo tipo ou compatíveis.

ATENÇÃO A STRINGS

O operador `==` compara referências de objetos, não conteúdo de strings. Use `.Equals()` para comparar strings.



CONDIÇÃO SIMPLES (IF)

Usado quando queremos executar um bloco de código apenas se uma condição for verdadeira.

```
if (condicao) {  
    // Código a ser executado se a condição for verdadeira;  
}
```

```
int idade = 18;  
if (idade >= 18) {  
    Console.WriteLine("Você é maior de idade.");  
}
```



CONDIÇÃO COMPOSTA (IF-ELSE)

Utilizado quando queremos executar um bloco de código se a condição for verdadeira e outro bloco se a condição for falsa.

```
if (condicao) {  
    // Código a ser executado se a condição for verdadeira  
} else {  
    // Código a ser executado se a condição for falsa  
}
```



```
int numero = 5;
if (numero % 2 == 0) {
    Console.WriteLine("O número é par.")
} else {
    Console.WriteLine("O número é ímpar.")
}
```



OPERADORES LÓGICOS EM C#

E (&&)

Retorna verdadeiro SOMENTE se ambos os valores forem verdadeiros.

OU (||)

Retorna verdadeiro se UM deles for verdadeiro.

NÃO (!)

Inverte o valor lógico do valor.



E (&&)

```
bool expressao1 = true;
bool expressao2 = false;

if (expressao1 && expressao2){
    Console.WriteLine("Ambas as expressões são verdadeiras.");
} else {
    Console.WriteLine("Pelo menos uma das expressões não é verdadeira.");
}
```



OU (||)

```
bool expressao1 = true;
bool expressao2 = false;

if (expressao1 || expressao2){
    Console.WriteLine("Pelo menos uma das expressões é verdadeira.");
} else {
    Console.WriteLine("Ambas as expressões são falsas.");
}
```




NÃO (!)

```
bool expressao = true;

if (!expressao){
    Console.WriteLine("A expressão é falsa.");
} else {
    Console.WriteLine("Ambas a expressão é verdadeira.");
}
```



TABELA VERDADE

p	q	$\sim p$	$\sim q$	$p \wedge q$	$p \vee q$
V	V	F	F	V	V
V	F	F	V	F	V
F	V	V	F	F	V
F	F	V	V	F	F



SWITCH CASE

Usado para simplificar múltiplas verificações em um valor.

Alternativa ao uso excessivo de declarações if-else.

```
switch (expressao)
{
    case valor1:
        //codigo a ser executado se expressao == valor1
        break;
    case valor2:
        //codigo a ser executado se expressao == valor2
        break;
    default:
        //codigo a ser executado se nenhum caso for valido
        break;
}
```



A expressão é avaliada uma vez.

O fluxo é redirecionado para o caso correspondente.

O break encerra o switch para evitar a execução de outros casos.

O bloco default é opcional e é executado se nenhum caso corresponder.



```
int diaSemana = 3;
string mensagem;

switch (diaSemana)
{
    case 1:
        mensagem = "Segunda-feira";
        break;
    case 2:
        mensagem = "Terça-feira";
        break;
    default:
        mensagem = "Dia inválido";
        break;
}
```



OPERADOR TERNÁRIO

```
condição ? valor_se_verdadeiro : valor_se_falso;
```

A condição é avaliada primeiro.

Se a condição for verdadeira, o valor após o ponto de interrogação é retornado.

Se a condição for falsa, o valor após os dois pontos é retornado.



```
int idade = 20;  
string status = (idade >= 18) ? "Adulto" : "Menor de idade";
```



VANTAGENS DO OPERADOR TERNÁRIO EM C#

Simplicidade:

Reduz a necessidade de escrever um bloco if-else completo.

Legibilidade:

Em situações simples, pode tornar o código mais limpo e claro.



LIMITAÇÕES E CONSIDERAÇÕES

COMPLEXIDADE:

Não é adequado para estruturas condicionais complexas.

LEGIBILIDADE:

Evite usar o operador ternário em expressões muito longas para não prejudicar a legibilidade.